

Phase 2 — Detailed Block Breakdown

The DOM is where JavaScript meets the page — entirely new territory for a Python developer. All topics map to **javascript.info Part 2** (Browser: Document, Events, Interfaces). Work through them in order over roughly one week; skip coordinates, scrolling and drag'n'drop for now.

BLOCK 8 The DOM & Browser APIs

~1 week

Day	Topic & javascript.info page	Python Dev Note
Day 1	DOM tree javascript.info/dom-nodes	The page is a tree of nodes — elements, text, even comments. No Python analogue; this is the new territory.
	Walking the DOM javascript.info/dom-navigation	<code>parentNode</code> , <code>children</code> , <code>nextElementSibling</code> . Traverse it like a family tree.
Day 2	■ Searching: <code>getElement*</code> , <code>querySelector*</code> javascript.info/searching-elements-dom	Your workhorses. CSS-selector syntax: <code>'#id'</code> , <code>'class'</code> , <code>'ul li'</code> . <code>querySelectorAll</code> returns a <code>NodeList</code> , not an array.
Day 3	Node properties: type, tag, contents javascript.info/basic-dom-node-properties	<code>textContent</code> vs <code>innerHTML</code> — the latter parses HTML, so it carries an XSS risk.
	Attributes and properties javascript.info/dom-attributes-and-properties	DOM properties and HTML attributes look alike but are not the same thing.
Day 4	Modifying the document javascript.info/modifying-document	<code>createElement</code> , <code>append</code> , <code>prepend</code> , <code>before</code> , <code>remove</code> — building and inserting nodes.
	Styles and classes javascript.info/styles-and-classes	Prefer <code>classList.add / remove / toggle</code> over poking <code>.style</code> directly.
Day 5	Introduction to browser events javascript.info/introduction-browser-events	<code>addEventListener('click', fn)</code> . <code>event.target</code> vs <code>event.currentTarget</code> trips people up.
	Bubbling and capturing javascript.info/bubbling-and-capturing	Events propagate up the tree — that upward travel is "bubbling".
Day 6	■ Event delegation (then start the milestone) javascript.info/event-delegation	One listener on the parent, dispatch by <code>event.target</code> — the clean way to handle a list that keeps changing. Key for your filter.

Milestone: Build the live search filter — type in an input, filter a visible list in real time. To exercise the whole block: render the list from a JS array with `createElement` (not hardcoded HTML), filter on the **input** event, and toggle a `.hidden` class rather than rebuilding the DOM on every keystroke. Then ship it to GitHub Pages with a live URL.

Notes for Python Developers Specifically

- Reach for the **input** event, not **keyup** — it fires correctly for paste, autofill and held keys too.
- Your Block 3 array methods carry straight over: **filter** to narrow the list, then render the results. The DOM is new, but the data logic is the Python you already know.
- Live in the DevTools console — select an element, then poke at `.textContent` and `.classList` directly. It is the fastest way to make the DOM feel concrete coming from Python.
- Blocks 4 (closures) and 6 (async) were the Phase 1 hard parts; here the whole block is new, so expect it to feel less like translating and more like genuinely fresh material — that is normal.